

TechTribers

Java Interview Series · Part 7 · Q305–Q339

JVM Internals, Memory & GC

Runtime memory areas, class loading, JIT, start-up, object layout, the Java Memory Model, every major garbage collector, and production troubleshooting, updated to JDK 27.

35	23	11	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `JV-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

35	5	23	11
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🏛️	01	JVM Architecture & Class Loading	Q305–Q311	4
⚡	02	Execution Engine & Start-up	Q312–Q316	10
🧱	03	Objects, References & the Memory Model	Q317–Q323	14
♻️	04	Garbage Collection	Q324–Q333	20
🩺	05	Troubleshooting the JVM in Production	Q334–Q339	27

© TechTribers. For personal use only. Do not redistribute.

All questions

🏛️ JVM Architecture & Class Loading

Q305 What are the JVM's runtime data areas? 4

Q306 What is the difference between StackOverflowError and OutOfMemoryError? 5

Q307 What replaced PermGen, and why? 6

Q308 What are the phases of class loading? 6

Q309 What is the difference between ClassNotFoundException and NoClassDefFoundError? 7

Q310 What is bytecode, and how do you inspect it? 8

Q311 What are the rules for the `main` method? Is `main` a keyword? 9

⚡ Execution Engine & Start-up

Q312 How does the JIT compiler optimise code? 10

Q313 What is escape analysis? 11

Q314 Why do Java services start slowly, and how can start-up and warm-up be improved? 11

Q315 GraalVM native image vs the JIT-based JVM: when would you choose each? 12

Q316 What is a safepoint, and what does stop-the-world mean? 13

🧱 Objects, References & the Memory Model

Q317 What does a Java object look like in memory, and what are compact object headers? 14

Q318 What are strong, soft, weak and phantom references? 15

Q319 Why is `finalize()` deprecated, and what replaces it? 16

Q320 What is the Java Memory Model (JMM)? 17

Q321 What is off-heap memory? How do direct buffers and the Foreign Function & Memory API work? 18

Q322 How does the JVM allocate objects so quickly? What is a TLAB? 18

Q323 What is G1 string deduplication? 19

♻️ Garbage Collection

Q324 How does garbage collection decide what to collect? 20

Q325 Explain the generational heap: young generation, old generation, minor and major GC. 21

Q326 What garbage collectors does HotSpot offer, and how do you choose? 21

Q327 How does the G1 garbage collector work? 22

Q328 What is ZGC, and what changed with Generational ZGC? 23

Q329 Should you call `System.gc()`? 24

Q330 How do you enable and read GC logs? 24

Q331 Which JVM memory and GC flags matter most in production? 25

Q332 What are throughput, latency and footprint in GC tuning? 26

Q333 What kinds of OutOfMemoryError exist, and how do you handle each? 26

🩺 Troubleshooting the JVM in Production

Q334 How do you take and analyse a thread dump? 27

Q335 How do you analyse a heap dump? What is retained size? 28

Q336 What is Java Flight Recorder (JFR), and why is it safe in production? 29

Q337 A Java service is at 100% CPU. How do you find the cause? 29

Q338 How does the JVM behave in containers? What is OOMKilled vs OutOfMemoryError? 30

Q339 What is the difference between a heap dump, a thread dump and a core dump? 31

© TechTribers. For personal use only. Do not redistribute.

Section 01

JVM Architecture & Class Loading

Where things live in memory, how classes get into the JVM, and the errors you see when that goes wrong.

Questions 305–311 · 7 questions



Q305 JV-001

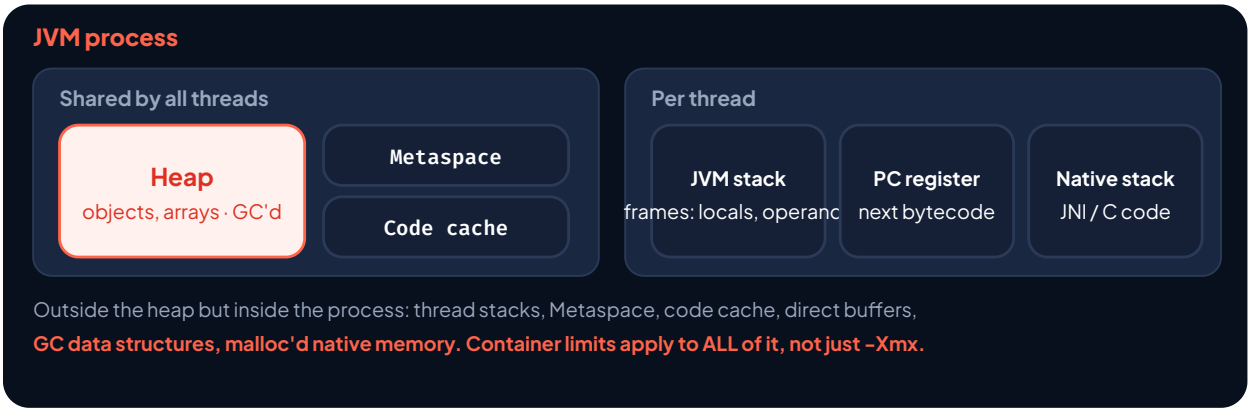
• Intermediate

What are the JVM's runtime data areas?

⚡ Quick answer

Shared by all threads: the **heap** (objects), **Metaspace** (class metadata) and the **code cache** (JIT-compiled code). Per thread: the **JVM stack** (frames with locals and operands), the **PC register** and the **native method stack**.

Area	Holds	Sized by	Error when full
Heap	All objects and arrays	`-Xms` , `-Xmx` , `MaxRAMPercentage`	`OutOfMemoryError: Java heap space`
Metaspace	Class metadata, method bytecode	`-XX:MaxMetaspaceSize` (unbounded by default)	`OutOfMemoryError: Metaspace`
Code cache	JIT-compiled native code	`-XX:ReservedCodeCacheSize`	JIT stops compiling (warning, slowdown)
JVM stack	One frame per method call	`-Xss` (default ~1 MB)	`StackOverflowError`



© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
"Static variables live on the stack" is wrong. Since Java 8, static fields live with the `Class` object on the heap; the class metadata lives in Metaspace.

Q306 JV-002 • Basic

What is the difference between StackOverflowError and OutOfMemoryError?

⚡ **Quick answer**
`StackOverflowError` means one thread's stack ran out of frames, almost always from unbounded recursion. `OutOfMemoryError` means the JVM couldn't allocate in a memory area (heap, Metaspace, native threads, direct memory) even after GC.

Both are `Error`s, not `Exception`s: they signal conditions an application generally can't recover from. Catching them, as in this demo, is acceptable only to log and shut down cleanly.

Java

```
static int depth;
static void recurse() { depth++; recurse(); }

try { recurse(); }
catch (StackOverflowError e) { System.out.println("StackOverflowError after deep recursion: " +
(depth > 1000)); }

List<long[]> hog = new ArrayList<>();
try { while (true) hog.add(new long[10_000_000]); }
catch (OutOfMemoryError e) { hog.clear(); System.out.println("OutOfMemoryError: " +
e.getMessage().startsWith("Java heap") + " / " + e.getClass().getSimpleName()); }
```

▶ **Output**

```
StackOverflowError after deep recursion: true
OutOfMemoryError: true / OutOfMemoryError
```

💡 **Pro tip**
Run production JVMs with `-XX:+HeapDumpOnOutOfMemoryError -XX:+ExitOnOutOfMemoryError` so you get evidence and a clean restart instead of a half-broken process.

© TechTribers. For personal use only. Do not redistribute.

Q307 JV-003

• Intermediate

What replaced PermGen, and why?

> ⚡ **Quick answer**
> Java 8 removed the Permanent Generation and moved class metadata to **Metaspace** in native memory, which grows automatically. Interned strings and static fields had already moved to the heap (Java 7/8).

Why PermGen was a problem:

- It had a fixed maximum (`-XX:MaxPermSize`), so apps with many classes (app servers, Groovy, heavy proxies) hit `OutOfMemoryError: PermGen space`.
- Tuning it was guesswork and it had to be collected with full GCs.

Metaspace is unbounded by default, sized by actual need, and its memory is freed when the class loader that loaded the classes becomes unreachable.

> ⚠️ **Interview trap**
> Unbounded doesn't mean leak-proof. A class-loader leak (e.g. redeploying a webapp while a thread still references the old loader) makes Metaspace grow until the container or OS kills the process. Set `-XX:MaxMetaspaceSize` in production.

Q308 JV-004

• Intermediate

What are the phases of class loading?

> ⚡ **Quick answer**
> **Loading** (find the bytes, create the `Class` object), **linking** (verification, preparation of static fields to defaults, resolution of symbolic references) and **initialization** (running static initializers), which happens lazily on first active use.

Java

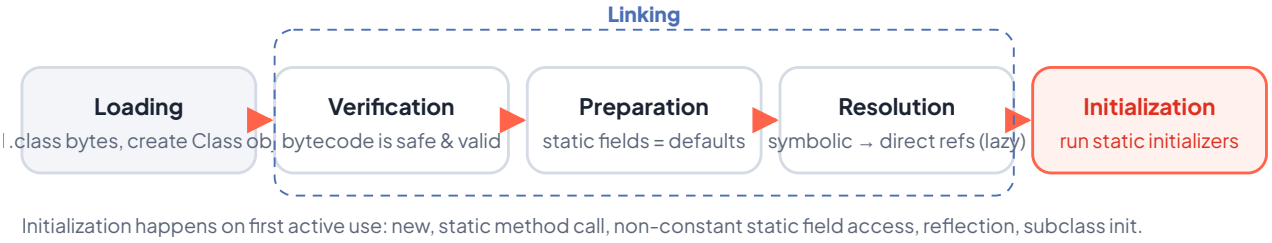
```
class Config {
    static final String NAME = "TT";              // compile-time constant: inlined, no init
    static int counter = log("static field");
    static { log("static block"); }
    static int log(String s) { System.out.println("init: " + s); return 1; }
}
System.out.println("1. " + Config.NAME);          // constant -> class NOT initialized
System.out.println("2. " + Config[].class.getSimpleName());   // array type -> no init
System.out.println("3. " + Config.counter);       // first active use -> init now
System.out.println("4. " + Config.counter);       // already initialized
```

▶ **Output**

```
1. TT
2. Config[]
init: static field
```


© TechTribers. For personal use only. Do not redistribute.

```
init: static block
3. 1
4. 1
```



⚠️ **Interview trap**

Accessing a `static final` compile-time constant does **not** initialise the class: the compiler copies the value into the caller's bytecode. Changing the constant without recompiling callers leaves them with the old value.

? Q309 JV-005

• Intermediate

What is the difference between ClassNotFoundException and NoClassDefFoundError?

⚡ **Quick answer**

`ClassNotFoundException` (checked) is thrown when code explicitly loads a class by name (`Class.forName`, `loadClass`) and it isn't found. `NoClassDefFoundError` (an `Error`) means a class that was present at compile time can't be loaded or initialised at runtime.

Common `NoClassDefFoundError` causes in production: a dependency missing from the runtime classpath (`provided` scope in Maven), version conflicts between jars, or, as shown, a failed static initializer, after which the class is permanently unusable in that class loader.

Java

```java
class Fragile {
    static final int VALUE = Integer.parseInt("not-a-number");   // static init will fail
}
try { Class.forName("com.techtribers.Missing"); }
catch (ClassNotFoundException e) { System.out.println("CNFE: " + e.getMessage()); }

try { System.out.println(Fragile.VALUE); }
catch (ExceptionInInitializerError e) { System.out.println("1st use: ExceptionInInitializerError");
}

try { System.out.println(Fragile.VALUE); }
catch (NoClassDefFoundError e) { System.out.println("2nd use: NoClassDefFoundError"); }
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
CNFE: com.techtribers.Missing
1st use: ExceptionInInitializerError
2nd use: NoClassDefFoundError
```

💡 **Pro tip**

When you see `NoClassDefFoundError: Could not initialize class X`, scroll up the logs: the real cause is the first `ExceptionInInitializerError`.

⚙️ **Q310** JV-006 • Intermediate

What is bytecode, and how do you inspect it?

⚡ **Quick answer**

Bytecode is the platform-independent instruction set in `.class` files: a stack-based language of ~200 opcodes (`iload`, `invokevirtual`, `getfield` …) executed by the JVM. Inspect it with `javap -c -p`.

Why it's worth knowing:

- Explaining `i++` races: `getfield`, `iconst_1`, `iadd`, `putfield` are separate instructions.
- Seeing what the compiler generates: string concatenation via `invokedynamic`, lambdas via `invokedynamic`, records' `equals` / `hashCode` via `ObjectMethods`.
- Libraries such as Spring, Hibernate, Mockito and Byte Buddy generate bytecode at runtime; Java 24's Class-File API (JEP 484) standardises this.

Terminal

```
// Source
public int add(int a, int b) { return a + b; }

// $ javap -c -p Calculator.class
//   public int add(int, int);
//     Code:
//        0: iload_1        // push a
//        1: iload_2        // push b
//        2: iadd           // pop both, push a + b
//        3: ireturn        // return top of stack
```

💡 **Pro tip**

The JVM runs much more than Java: Kotlin, Scala, Groovy and Clojure all compile to the same bytecode, which is why they interoperate with Java libraries.

© TechTribers. For personal use only. Do not redistribute.

Q311 JV-007

• Basic

What are the rules for the `main` method? Is `main` a keyword?

> ⚡ **Quick answer**
> `main` is an ordinary identifier, not a keyword. The classic launcher entry point is `public static void main(String[] args)`; since Java 25 (JEP 512) a compact source file may use an instance `void main()` with no `public`, `static` or parameter.

Other facts interviewers probe:

- `String... args` (varargs) and `final String[] args` are also valid signatures.
- The launcher only calls the matching signature; other overloads are normal methods.
- `int main` or a missing `static` (before Java 25's rules) gives "main method not found" at launch, not a compile error.

Java

```java
public class EntryPoints {
    public static void main(String[] args) {            // classic: works on every Java version
        System.out.println("args = " + args.length);
        main("overload");                                // overloading main is legal
    }
    static void main(String s) { System.out.println("overloaded main(" + s + ")"); }
}

// Java 25+ compact source file (Hello.java), run with: java Hello.java
// void main() {
//     IO.println("Hello, TechTribers");
// }
```

▶ **Output**

```
args = 0
overloaded main(overload)
```


© TechTribers. For personal use only. Do not redistribute.

Section 02

Execution Engine & Start-up

How bytecode becomes fast machine code, why Java needs warm-up, and the modern ways to start faster.

Questions 312–316 · 5 questions

Q312 JV-008

• Advanced

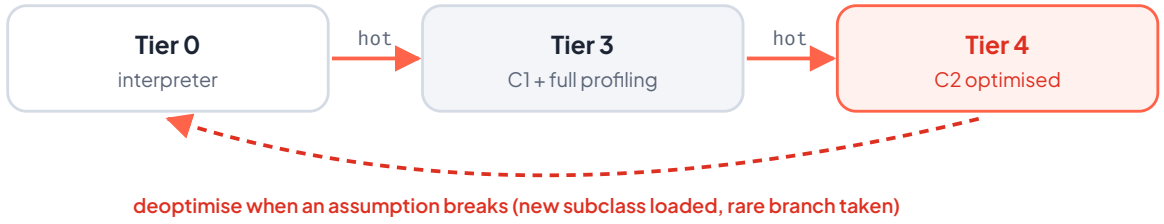
How does the JIT compiler optimise code?

⚡ Quick answer

HotSpot starts in the interpreter while profiling; hot methods are compiled by C1 (quick) and then C2 (aggressive) using the collected profile. C2 inlines calls, removes dead code, unrolls loops, eliminates allocations via escape analysis, and deoptimises if its assumptions break.

Key optimisations:

- **Inlining**: copies small or hot callees into the caller, the gateway to every other optimisation.
- **Devirtualisation**: if only one implementation of an interface is loaded, calls become direct (and are deoptimised if another appears).
- **Intrinsics**: hand-tuned machine code for methods like `System.arraycopy`, `String.equals`, `Math` and CRC32.
- **Loop optimisations**: unrolling, range-check elimination, auto-vectorisation (SIMD).



C2 tricks: inlining · escape analysis · loop unrolling · dead-code elimination · intrinsics (e.g. String.equals, Math)

💡 Pro tip

This is why micro-benchmarks need JMH: a naive timing loop measures the interpreter, then C1, then C2 with half the code eliminated as dead.

🎯 Likely follow-up

See it happen with `-XX:+PrintCompilation`, or with JITWatch on a `-XX:+LogCompilation` log.

© TechTribers. For personal use only. Do not redistribute.

Q313 JV-009

• Advanced

What is escape analysis?

⚡ Quick answer

The JIT checks whether an object can escape the method or thread that created it. If not, it can allocate it on the stack or break it into plain local variables (**scalar replacement**) and remove locking on it, eliminating the heap allocation entirely.

Objects **escape** when they are returned, stored in a field or collection, passed to a method that isn't inlined, or published to another thread. Escape analysis is on by default (`-XX:+DoEscapeAnalysis`).

Java

```
record Point(int x, int y) { }

static long distanceSquared(int x1, int y1, int x2, int y2) {
    Point a = new Point(x1, y1);          // never escapes this method
    Point b = new Point(x2, y2);
    long dx = a.x() - b.x(), dy = a.y() - b.y();
    return dx * dx + dy * dy;             // after C2: no Point objects allocated at all
}
long total = 0;
for (int i = 0; i < 1_000_000; i++) total += distanceSquared(i, i, 0, 0) % 7;
System.out.println(total > 0);
```

▶ Output

```
true
```

⚠️ Interview trap

Don't contort code to "help" escape analysis or avoid small allocations prematurely. Short-lived objects are cheap; measure with a profiler's allocation view before optimising.

Q314 JV-010

• Advanced

Why do Java services start slowly, and how can start-up and warm-up be improved?

⚡ Quick answer

At start-up the JVM must load, verify and link thousands of classes, and peak speed only arrives after the JIT has profiled and compiled hot code. Fixes range from class-data sharing and the AOT cache (JDK 24+) to CRaC snapshots and GraalVM native images.

Technique	What it saves	Trade-off
CDS / AppCDS	Class parsing and verification	Archive per app build

© TechTribers. For personal use only. Do not redistribute.

AOT cache (JEP 483, 514, 515)	Class loading, linking and method profiles from a training run	Training run in CI; JDK 24/25+
CRaC (checkpoint/restore)	Everything: restores a warmed-up process	Linux only, careful resource handling
GraalVM native image	Start in milliseconds, small memory	Closed world: reflection needs config, slower peak in some cases
Lazy init, fewer beans	Framework start-up work	First requests pay the cost

Terminal

```
# JDK 25: create and use an AOT cache in two commands (JEP 514)
java -XX:AOTCacheOutput=app.aot -jar app.jar     # training run: exercise typical requests, then exit
java -XX:AOTCache=app.aot -jar app.jar           # production: faster start-up and warm-up
```

💡 **Pro tip**

For Kubernetes, a slow start mostly hurts autoscaling and rollouts. Combine an AOT cache with a readiness probe that waits for warm-up (or sends warm-up traffic) so cold pods don't serve real users.

Q315 JV-011 • Advanced

GraalVM native image vs the JIT-based JVM: when would you choose each?

⚡ **Quick answer**

Native image compiles the app ahead of time into a standalone executable: start-up in milliseconds and low memory, but a "closed world" (reflection, proxies and resources must be declared). The JIT JVM starts slower but adapts to real traffic and usually reaches higher peak throughput.

Choose **native image** for: serverless functions, CLIs, scale-to-zero services, and memory-dense deployments. Spring Boot 3, Quarkus and Micronaut support it directly.

Choose the **JVM** for: long-running, throughput-heavy services, apps with heavy dynamic behaviour (runtime class generation, plugins), and when you need full JFR/agent tooling.

⚠️ **Interview trap**

Native images don't run your test suite's assumptions automatically: code paths using reflection that worked on the JVM can fail only at runtime in the native binary. Run integration tests against the native build in CI.

© TechTribers. For personal use only. Do not redistribute.

Q316 JV-012 • Advanced

What is a safepoint, and what does stop-the-world mean?

> ⚡ **Quick answer**
> A safepoint is a point where every Java thread is paused in a known state so the JVM can safely inspect or change it (GC phases, deoptimisation, thread dumps, class redefinition). A stop-the-world pause is the time all application threads are held at a safepoint.

Threads check for a pending safepoint at method returns and loop back-edges. **Time-to-safepoint** can dominate a pause if one thread runs a long loop without a poll (historically counted `int` loops, now mostly fixed by loop strip mining).

Modern collectors (ZGC, Shenandoah, much of G1) do most work concurrently, so their stop-the-world pauses are short, but they still need brief safepoints.

Terminal

```
# Log safepoint operations and their durations
java -Xlog:safepoint*=info -jar app.jar
# [info][safepoint] Safepoint "G1CollectForAllocation", Time since last: 120 ms,
#                   Reaching safepoint: 41 us, At safepoint: 3.2 ms, Total: 3.3 ms
```

> 💡 **Pro tip**
> If GC logs show short pauses but users see long freezes, check safepoint logs: a slow time-to-safepoint or frequent non-GC safepoints (e.g. biased-lock revocation in older JDKs) can be the culprit.

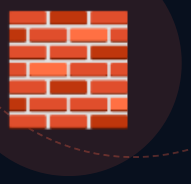
© TechTribers. For personal use only. Do not redistribute.

Section 03

Objects, References & the Memory Model

What an object costs, how reachability strength controls collection, and the rules that make concurrent code correct.

Questions 317–323 · 7 questions



Q317 JV-013

• Advanced

What does a Java object look like in memory, and what are compact object headers?

⚡ Quick answer

Each object has a header (a **mark word** for hash, lock and GC age, plus a **class pointer**), then its fields, padded to 8 bytes. Compact object headers (JEP 450 → 519, default in JDK 27) shrink the header from 12 to 8 bytes on 64-bit JVMs.

Typical sizes on a 64-bit JVM with compressed pointers:

- Empty object: 16 bytes with classic headers (12-byte header plus 4 bytes of padding).
- `Integer`: 16 bytes for 4 bytes of data, a 4× overhead that explains why `List<Integer>` is so much bigger than `int[]`.
- Array: header plus a 4-byte length, then the elements.

Compressed oops store 64-bit references in 32 bits (shifted by 3), which works for heaps up to ~32 GB. Above that, every reference doubles in size, so a 34 GB heap can hold *less* than a 31 GB one.

64-bit HotSpot object layout: new Point(1, 2) with two int fields

Classic (compressed oops)



Compact headers (JEP 519, default in JDK 27)



Objects are 8-byte aligned. Smaller headers = more objects per cache line and less heap for the same data.

⚠️ Interview trap

Crossing the ~32 GB line silently disables compressed oops. If you need a big heap, either stay below 31 GB or go well above (48 GB+).

© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

Measure exact layouts with JOL (Java Object Layout): `ClassLayout.parseClass(Point.class).toPrintable()`.

Q318 JV-014 • Advanced

What are strong, soft, weak and phantom references?

⚡ **Quick answer**

They define how strongly an object is reachable. **Strong**: never collected while reachable. **Soft**: cleared when memory is tight. **Weak**: cleared at the next GC once only weakly reachable. **Phantom**: enqueued after the object is collected, for cleanup actions.

Java

```
Object big = new byte[1024];
WeakReference<Object> weak = new WeakReference<>(big);
System.out.println("strongly reachable: " + (weak.get() != null));

big = null;                                  // drop the only strong reference
System.gc();                                 // a request, but HotSpot honours it by default
Thread.sleep(100);
System.out.println("after GC, weak cleared: " + (weak.get() == null));

WeakHashMap<Object, String> meta = new WeakHashMap<>();
Object key = new Object();
meta.put(key, "session data");
key = null; System.gc(); Thread.sleep(100);
System.out.println("WeakHashMap entries: " + meta.size());
```

▶ **Output**

```
strongly reachable: true
after GC, weak cleared: true
WeakHashMap entries: 0
```

Strong	`Object o = new X()`	never while reachable
Soft	`SoftReference`	cleared under memory pressure
Weak	`WeakReference`	cleared at the next GC
Phantom	`PhantomReference`	after finalisation, for cleanup

Reachability weakens top to bottom. WeakHashMap keys, caches and Cleaner are built on these classes.

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`SoftReference`-based caches look clever but behave badly: under pressure the GC clears everything at once, causing cache stampedes, and soft references make GC work harder. Use a bounded cache (Caffeine) with explicit size or TTL limits.

Q319 JV-015

• Intermediate

Why is `finalize()` deprecated, and what replaces it?

⚡ **Quick answer**

Finalizers run at an unpredictable time (or never), on an unspecified thread, can resurrect objects, and slow down GC. They are deprecated for removal (JEP 421, Java 18). Use **try-with-resources** for deterministic cleanup, and `java.lang.ref.Cleaner` as a safety net.

Java

```
final class NativeBuffer implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();
    private final Cleaner.Cleanable cleanable;

    NativeBuffer(String name) {
        Runnable release = () -> System.out.println("released " + name);   // must NOT reference
'this'
        cleanable = CLEANER.register(this, release);
    }
    @Override public void close() { cleanable.clean(); }                  // runs at most once
}
try (NativeBuffer buf = new NativeBuffer("buf-1")) {
    System.out.println("using buffer");
}
```

▶ **Output**

```
using buffer
released buf-1
```

⚠️ **Interview trap**

If the cleanup action captures `this` (e.g. a lambda calling an instance method), the object stays reachable from the Cleaner forever and is never collected.

© TechTribers. For personal use only. Do not redistribute.

Q320 JV-016 • Advanced

What is the Java Memory Model (JMM)?

> ⚡ **Quick answer**
> The JMM defines when a write by one thread is guaranteed to be visible to a read in another, through **happens-before** relationships. Without one, the compiler, JIT and CPU may reorder or cache operations, so another thread may see stale or partially updated values.

The happens-before rules you'll use daily:

- **Program order**: within one thread, each action happens-before the next.
- **Monitor lock**: unlocking a monitor happens-before every later lock of the same monitor.
- **volatile**: a write to a volatile field happens-before every later read of it.
- **Thread start/join**: `t.start()` happens-before anything in `t`; everything in `t` happens-before `t.join()` returns.
- **final fields**: values set in the constructor are visible to all threads once the constructor finishes (if `this` didn't escape).
- **Transitivity**: if A → B and B → C, then A → C.

Java

```
class Config {
    private Map<String, String> settings;           // plain field
    private volatile boolean ready;                  // the publication flag

    void load() { settings = Map.of("region", "ap-south-1"); ready = true; }       // writer
    String region() { return ready ? settings.get("region") : null; }             // reader
}
// volatile write of 'ready' happens-before the volatile read that sees true,
// so the reader is guaranteed to see the fully built 'settings' map.
Config c = new Config();
Thread w = new Thread(c::load); w.start(); w.join();
System.out.println(c.region());
```

▶ Output

```
ap-south-1
```

> ⚠️ **Interview trap**
> Data races in Java are not undefined behaviour as in C++, but results are still surprising: without a happens-before edge, a loop reading a non-volatile flag may never see it change.

© TechTribers. For personal use only. Do not redistribute.

Q321 JV-017

• Advanced

What is off-heap memory? How do direct buffers and the Foreign Function & Memory API work?

⚡ Quick answer

Off-heap memory is native memory outside the GC-managed heap. `ByteBuffer.allocateDirect` gives NIO direct buffers (used by Netty and Kafka for zero-copy I/O). The FFM API (final in Java 22) provides safe, deterministic native memory via `Arena` and `MemorySegment`, and calls C functions without JNI.

Why go off-heap: huge caches without GC cost, zero-copy I/O with the OS, and sharing memory with native libraries. The price is manual lifetime management and memory that heap dumps don't show.

Java

```
// Direct NIO buffer: native memory, freed when the buffer object is collected
ByteBuffer direct = ByteBuffer.allocateDirect(1024);
direct.putInt(42).flip();
System.out.println("direct buffer: " + direct.isDirect() + " value=" + direct.getInt());

// FFM API (Java 22+): explicit lifetime, freed when the arena closes
try (Arena arena = Arena.ofConfined()) {
    MemorySegment seg = arena.allocate(ValueLayout.JAVA_LONG, 3);
    for (int i = 0; i < 3; i++) seg.setAtIndex(ValueLayout.JAVA_LONG, i, (i + 1) * 100L);
    System.out.println("segment sum: " + (seg.getAtIndex(ValueLayout.JAVA_LONG, 0) +
seg.getAtIndex(ValueLayout.JAVA_LONG, 2)));
}
```

▶ Output

```
direct buffer: true value=42
segment sum: 400
```

⚠️ Interview trap

Direct buffers are freed only when their small heap wrapper is garbage-collected. A service with a large, quiet heap can run out of direct memory (`OutOfMemoryError: Direct buffer memory`) long before GC runs.

Q322 JV-018

• Advanced

How does the JVM allocate objects so quickly? What is a TLAB?

⚡ Quick answer

Each thread gets a **Thread-Local Allocation Buffer** (TLAB), a private chunk of Eden. Allocation is just a pointer bump inside the TLAB with no locking, so `new` typically costs a few nanoseconds; only when the TLAB fills does the thread grab a new one from the shared heap.

Why this matters:

© TechTribers. For personal use only. Do not redistribute.

- It explains why short-lived objects are cheap: allocation is a bump, and young GC cost depends on survivors, not on garbage.
- Large objects that don't fit a TLAB are allocated more slowly in shared Eden (or as humongous regions in G1).
- Allocation **rate** (MB/s) still drives GC frequency; JFR's allocation profiling shows which code paths allocate most.

Terminal

```
# Show TLAB statistics and allocation behaviour
java -Xlog:gc+tlab=debug -jar app.jar
# JFR events: jdk.ObjectAllocationInNewTLAB, jdk.ObjectAllocationOutsideTLAB,
jdk.ObjectAllocationSample
```

> 💡 **Pro tip**
> When someone suggests object pooling to "avoid allocation", TLABs are the counter-argument: pooling often costs more (synchronisation, retained memory, older objects) than allocating fresh.

🔁 **Q323** JV-019 • Intermediate

What is G1 string deduplication?

> ⚡ **Quick answer**
> With `-XX:+UseStringDeduplication`, the GC finds `String` objects with identical contents and makes them share one backing `byte[]`, cutting memory in apps holding many duplicate strings (JSON field values, DB rows). Unlike `intern()`, it's automatic and doesn't change identity.

Facts worth knowing:

- Only strings that survive a few young collections are considered, so short-lived strings cost nothing.
- Available for G1 since Java 8u20, and for Serial, Parallel, ZGC and Shenandoah since Java 18.
- The `String` objects stay distinct (`==` still false); only their character arrays are shared.

Terminal

```
java -XX:+UseG1GC -XX:+UseStringDeduplication -Xlog:stringdedup*=debug -jar app.jar
# [gc,stringdedup] Concurrent String Deduplication 2650.3K->1210.4K(1439.9K) avg 54.4%
```

> 💡 **Pro tip**
> Check first whether duplicate strings are actually your problem: MAT's "Group By Value" on `java.lang.String` shows the duplicate counts in a heap dump.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Garbage Collection

How collectors find garbage, why the heap is split into generations, and how to pick, tune and read a GC.

Questions 324–333 · 10 questions



Q324 JV-020 • Intermediate

How does garbage collection decide what to collect?

⚡ Quick answer

Tracing GC: start from **GC roots** (thread stacks' local variables, static fields, JNI references, active monitors), mark everything reachable, and reclaim everything else. Reference counting isn't used, so cycles of unreachable objects are collected normally.

The classic phases:

- **Mark**: traverse the object graph from the roots.
- **Sweep**: reclaim unmarked memory (leaves fragmentation).
- **Compact / copy**: move live objects together so allocation is a fast pointer bump.

Modern collectors do marking (and for ZGC/Shenandoah, compaction) **concurrently** with the application, using read/write barriers to track changes made while they work.

Java

```
class Node { Node next; }
Node a = new Node(), b = new Node();
a.next = b; b.next = a;                // a cycle
a = null; b = null;                    // unreachable from any root: both collectable
System.out.println("cycle is garbage despite references to each other");
```

▶ Output

```
cycle is garbage despite references to each other
```

⚠️ Interview trap

"Objects are collected when their reference count reaches zero" is the most common wrong answer. Java uses reachability from roots, which is exactly why cycles aren't leaks.

© TechTribers. For personal use only. Do not redistribute.

Q325 JV-021

• Intermediate

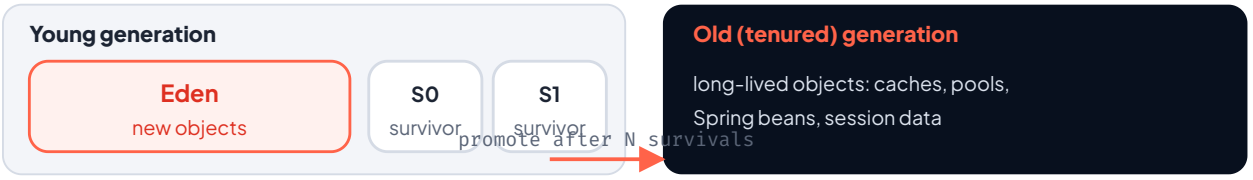
Explain the generational heap: young generation, old generation, minor and major GC.

⚡ Quick answer

Based on the weak generational hypothesis (most objects die young), new objects go to **Eden**; survivors of minor GCs move between **survivor** spaces and are promoted to the **old generation** after enough collections. Minor GC is frequent and cheap; full GC of the whole heap is the expensive one to avoid.

Collection	Scope	Typical cost
Minor (young)	Eden + survivors	Milliseconds; proportional to surviving objects
Mixed (G1)	Young + selected old regions	Bounded by the pause target
Major / old	Old generation	Concurrent in G1, ZGC, Shenandoah
Full GC	Entire heap, stop-the-world	Seconds on large heaps; a red flag in modern JVMs

Generational heap: most objects die young, so collect the young space often and cheaply



Minor GC: copy live objects Eden → survivor → old (pause short, cost ∝ LIVE objects, not garbage)

Major / mixed GC: also reclaims old regions · Full GC: whole heap, the expensive one to avoid

💡 Pro tip

Because young-GC cost depends on **live** objects, creating lots of short-lived garbage is cheap. What hurts is "medium-lived" data that survives a few collections and then dies in the old generation, such as oversized request-scoped caches.

Q326 JV-022

• Advanced

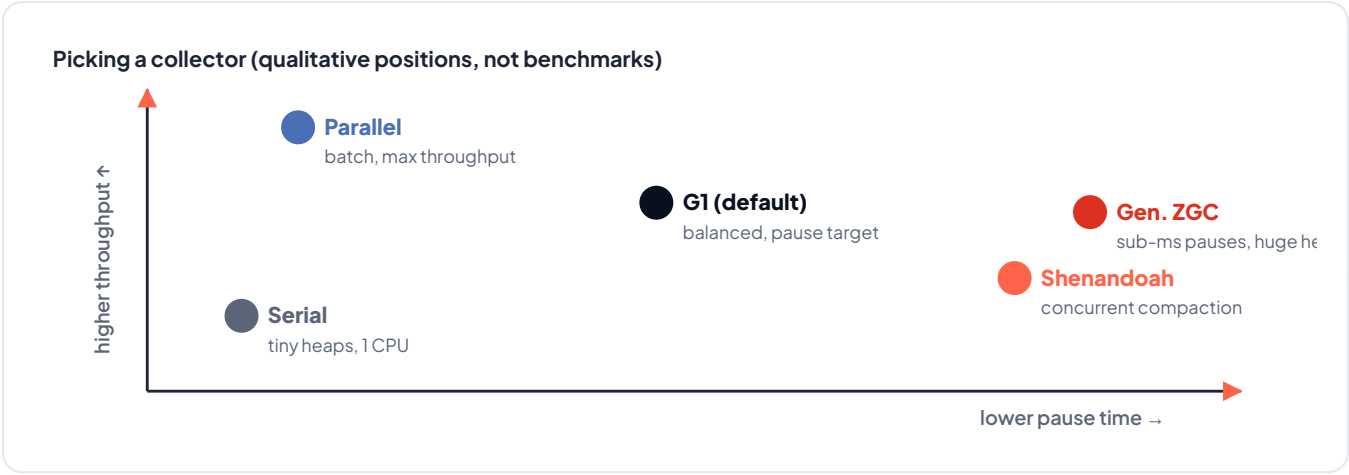
What garbage collectors does HotSpot offer, and how do you choose?

⚡ Quick answer

Serial (single thread, tiny heaps), Parallel (max throughput, longer pauses), **G1** (default: regions, pause-time target), **ZGC** (generational since 21, sub-millisecond pauses on huge heaps), Shenandoah (concurrent compaction) and Epsilon (no GC, for testing).

© TechTribers. For personal use only. Do not redistribute.

Collector	Flag	Best for
Serial	`-XX:+UseSerialGC`	Small heaps, single-CPU containers, CLIs
Parallel	`-XX:+UseParallelGC`	Batch jobs where throughput beats latency
G1	`-XX:+UseG1GC` (default)	General-purpose services, 2–64 GB heaps
ZGC	`-XX:+UseZGC`	Latency-critical services, very large heaps
Shenandoah	`-XX:+UseShenandoahGC`	Low pause on OpenJDK builds that include it
Epsilon	`-XX:+UseEpsilonGC`	Benchmarks, short-lived jobs that never fill the heap



💡 **Pro tip**

Before JDK 27, a JVM that saw fewer than 2 CPUs or under ~1.8 GB of memory silently chose Serial GC, which surprised many containerised apps. JDK 27 makes G1 the default in all environments. On older JDKs, set the collector explicitly.

🧩 **Q327** `JV-023` • Advanced

How does the G1 garbage collector work?

⚡ **Quick answer**

G1 divides the heap into equal regions that can be Eden, Survivor, Old or Humongous. It collects young regions in short pauses, marks the old generation concurrently, then runs **mixed** collections that evacuate the old regions with the most garbage first, aiming for `-XX:MaxGCPauseMillis` (default 200 ms).

Key details:

- **Remembered sets** track pointers into each region, so a region can be collected without scanning the whole heap.

© TechTribers. For personal use only. Do not redistribute.

- **Humongous objects** (at least half a region) get contiguous regions of their own; lots of them (big byte arrays) cause fragmentation and premature GCs.
- **Evacuation failure** (no free regions to copy into) falls back to an expensive full GC; a bigger heap or lower `InitiatingHeapOccupancyPercent` helps.

Terminal

```
# Common, minimal G1 tuning: set heap and pause goal, let G1 adapt the rest
java -Xms4g -Xmx4g -XX:+UseG1GC -XX:MaxGCPauseMillis=100 \
     -Xlog:gc*:file=gc.log:time,uptime,level,tags -jar app.jar
```

G1: heap split into equal regions (1–32 MB); each region's role changes over time



Mixed GCs collect young regions + the old regions with the most garbage first ("Garbage First").

⚠️ **Interview trap**

Setting the young generation size (`-Xmn` , `NewRatio`) with G1 disables its adaptive sizing, so it can no longer meet the pause target. Tune the pause goal, not the generation sizes.

⚡ **Q328** JV-024 • Advanced

What is ZGC, and what changed with Generational ZGC?

⚡ **Quick answer**

ZGC is a concurrent, region-based, compacting collector whose pauses stay below a millisecond regardless of heap size (up to 16 TB). Generational ZGC (Java 21) adds young/old generations for much better throughput; it became the default ZGC mode in Java 23, and non-generational ZGC was removed in Java 24.

How it stays pause-free: **coloured pointers** (metadata bits in references) and **load barriers** let ZGC relocate objects while the application runs; a thread touching a moved object fixes the reference on the fly.

When to choose it: latency SLOs in the low milliseconds (trading, gaming, real-time APIs), large heaps, or when G1's pauses show up in your p99 latency.

© TechTribers. For personal use only. Do not redistribute.

```
Terminal
java -XX:+UseZGC -Xmx16g -jar app.jar          # Java 23+: generational by default
java -XX:+UseZGC -XX:+ZGenerational -jar app.jar # Java 21-22: opt in explicitly
```

⚠️ **Interview trap**
ZGC uses more CPU and some extra memory than G1 for the same work. If throughput or cost per request matters more than tail latency, G1 can still be the better choice. Measure both.

Q329 JV-025 • Intermediate

Should you call `System.gc()`?

⚡ **Quick answer**
Almost never. It's only a request; on HotSpot it triggers a full stop-the-world GC by default (or a concurrent cycle with `-XX:+ExplicitGCInvokesConcurrent`). Calling it in application code adds long pauses and second-guesses a collector that knows better.

Legitimate niches: benchmarks and tests that want a clean heap, or tools measuring memory. Some libraries call it internally (old RMI did, every hour), which is why `-XX:+DisableExplicitGC` exists.

Be careful with that flag: some direct-buffer frameworks rely on `System.gc()` to release native memory when `MaxDirectMemorySize` is reached, so disabling it can cause `OutOfMemoryError: Direct buffer memory`.

💡 **Pro tip**
If you feel you need `System.gc()`, the real problem is usually heap sizing, a leak, or object churn. Profile allocation instead.

Q330 JV-026 • Intermediate

How do you enable and read GC logs?

⚡ **Quick answer**
Use unified logging (Java 9+): `-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=20m`. Each line shows the GC type, cause, heap before → after (capacity), and pause time. Analyse trends with tools like GCeasy or GCViewer.

Reading the example:

- **Line 1**: healthy young GC; 612 MB → 138 MB in under 10 ms.
- **Line 2**: triggered by a humongous allocation; the heap barely shrinks, so live data is high.

© TechTribers. For personal use only. Do not redistribute.

- **Line 3**: a 2.4-second full GC that freed almost nothing: the live set is near the heap size. That's a leak or an undersized heap, not a GC tuning problem.

Terminal

```
# Enable rotating GC logs (safe to leave on in production)
java -Xlog:gc*:file=/logs/gc.log:time,uptime,level,tags:filecount=5,filesize=20m -jar app.jar

# Typical G1 lines
# [2026-10-14T10:15:02.118+0530][12.4s][info][gc] GC(41) Pause Young (Normal) (G1 Evacuation Pause) 612M->138M(1024M) 9.812ms
# [2026-10-14T10:15:40.004+0530][50.3s][info][gc] GC(57) Pause Young (Concurrent Start) (G1 Humongous Allocation) 701M->650M(1024M) 15.1ms
# [2026-10-14T10:17:12.551+0530][142.8s][info][gc] GC(90) Pause Full (G1 Compaction Pause) 1019M->998M(1024M) 2418.6ms
```

> 💡 **Pro tip**
> Watch the heap **after** GC over time. A steadily rising floor across old-generation collections is the signature of a memory leak.

Q331 JV-027 • Advanced

Which JVM memory and GC flags matter most in production?

> ⚡ **Quick answer**
> Size the heap (`-Xms` / `-Xmx` , or `-XX:MaxRAMPercentage` in containers), pick the collector, set a pause goal, and turn on evidence: GC logging, heap dump on OOM, and JFR. Leave most other knobs at their defaults.

Flag families: **standard** options (`-cp` , `-D`) are stable; `-X` **options (** `-Xmx` , `-Xss` **) are non-standard but widely supported;** `-XX` options are HotSpot-specific and some need `-XX:+UnlockExperimentalVMOptions` or `UnlockDiagnosticVMOptions` .

Terminal

```
java \
  -XX:MaxRAMPercentage=75 -XX:InitialRAMPercentage=75 \   # heap from the container limit
  -XX:+UseG1GC -XX:MaxGCPauseMillis=150 \
  -XX:MaxMetaspaceSize=256m -Xss512k \                    # bound non-heap growth
  -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps \
  -XX:+ExitOnOutOfMemoryError \                           # let Kubernetes restart cleanly
  -Xlog:gc*:file=/logs/gc.log:time,uptime:filecount=5,filesize=20m \
  -XX:StartFlightRecording=disk=true,maxsize=250m,maxage=6h \
  -jar app.jar

# See every flag's effective value
java -XX:+PrintFlagsFinal -version | grep -E "MaxHeapSize|UseG1GC|MaxRAMPercentage"
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Copy-pasted flag lists from old blog posts (`-XX:+UseConcMarkSweepGC` , `-XX:MaxPermSize` , `-XX:+AggressiveOpts`) now stop the JVM from starting, because those options were removed.

📈 **Q332** `JV-028` • Intermediate

What are throughput, latency and footprint in GC tuning?

⚡ **Quick answer**
The three competing goals: **throughput** (share of time doing application work), **latency** (pause lengths, especially p99), and **footprint** (memory used). Improving one usually costs another, so pick the goal that matches the workload before tuning.

Workload	Primary goal	Typical choice
Nightly batch / ETL	Throughput	Parallel GC, big heap
REST API with SLOs	Latency (p99)	G1 with pause goal, or ZGC
Many small pods	Footprint	G1 or Serial, smaller heap, AOT/CDS
Low-latency trading	Latency (p99.9)	ZGC, pre-touched heap, minimal allocation

💡 **Pro tip**
A bigger heap almost always improves throughput and reduces GC frequency, but can make individual pauses longer with non-concurrent collectors. More memory is the cheapest GC tuning there is.

🧯 **Q333** `JV-029` • Advanced

What kinds of OutOfMemoryError exist, and how do you handle each?

⚡ **Quick answer**
The message tells you which memory ran out: **Java heap space**, **GC overhead limit exceeded** (Parallel GC), **Metaspace**, **unable to create native thread**, **Direct buffer memory**, or **requested array size exceeds VM limit**. Each has a different root cause and fix.

First response for any heap-related OOM:

- Get the heap dump (`-XX:+HeapDumpOnOutOfMemoryError`) and the GC log.
- In Eclipse MAT, open the **Leak Suspects** report and the dominator tree.
- Decide: is it a **leak** (live set grows forever) or **undersized** (live set is stable but bigger than the heap)? Fix the code in the first case, the sizing in the second.

© TechTribers. For personal use only. Do not redistribute.

`Java heap space`	live data > -Xmx, or a leak	heap dump + MAT
`GC overhead limit exceeded`	>98% time in GC, <2% freed	same as heap space
`Metaspace`	too many classes / classloader leak	-XX:MaxMetaspaceSize, redeploy leak
`unable to create native thread`	OS thread / memory limit hit	bounded pools, virtual threads
`Direct buffer memory`	off-heap NIO buffers exhausted	-XX:MaxDirectMemorySize, Netty leaks
`Requested array size exceeds VM limit`	array > ~2^31 elements	fix the algorithm

⚠️ **Interview trap**

Catching `OutOfMemoryError` and continuing leaves the JVM in an undefined state: some threads may have died mid-operation and locks may be inconsistent. Log, dump, and exit.

Section 05

Troubleshooting the JVM in Production

Thread dumps, heap dumps, flight recordings, CPU spikes and containers: the incident-response skills senior interviews test.

Questions 334–339 · 6 questions



Q334 JV-030 • Advanced

How do you take and analyse a thread dump?

⚡ **Quick answer**

Capture 3–5 dumps a few seconds apart with `jcmd <pid> Thread.print` (or `jstack`, or `kill -3`). Compare them: threads stuck in the same frame across dumps are your problem, whether `BLOCKED` on a lock, `WAITING` on a pool or I/O, or `RUNNABLE` in a hot loop.

Patterns to recognise:

- Many threads **BLOCKED** on the same lock → contention; find the owner and what it's doing (often I/O under a lock).
- Every pool thread **WAITING** in `HikariPool.getConnection` → connection-pool exhaustion.
- `Found one Java-level deadlock` → printed for you at the end of the dump.
- Threads **RUNNABLE** in the same method in every dump → CPU hotspot.

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
jcmd 4321 Thread.print > dump-1.txt; sleep 5; jcmd 4321 Thread.print > dump-2.txt

# "http-nio-8080-exec-17" #63 daemon prio=5 tid=0x00007f... nid=0x2c4f waiting for monitor entry
#    java.lang.Thread.State: BLOCKED (on object monitor)
#     at com.tt.billing.InvoiceService.next(InvoiceService.java:88)
#     - waiting to lock <0x000000071a2b3c40> (a java.lang.Object)
#     owned by "scheduler-1" #41
#
# "scheduler-1" #41 prio=5 ... runnable
#    java.lang.Thread.State: RUNNABLE
#     at java.net.SocketInputStream.socketRead0(Native Method)   <- holds the lock during I/O!
```

💡 **Pro tip**

For virtual threads, use `jcmd <pid> Thread.dump_to_file -format=json out.json` : the classic dump only shows platform threads.

Q335 JV-031 • Advanced

How do you analyse a heap dump? What is retained size?

⚡ **Quick answer**

Take a dump (`jcmd <pid> GC.heap_dump /tmp/app.hprof`), open it in Eclipse MAT or VisualVM, and sort the **dominator tree** by **retained size**: the memory that would be freed if that object were collected. Then follow the path to GC roots to see who keeps it alive.

Shallow vs retained:

- **Shallow size**: the object itself (header + fields).
- **Retained size**: the object plus everything reachable *only* through it.

A `HashMap` with a 48–byte shallow size can retain gigabytes. Leak hunting means finding the few objects with a huge retained size and asking why they're still referenced.



© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
A heap dump pauses the JVM and writes a file roughly the size of the used heap. On a 16 GB production heap that's a long pause plus disk pressure; take it from a pod removed from the load balancer if you can.

✈️ **Q336** JV-032 • Advanced

What is Java Flight Recorder (JFR), and why is it safe in production?

⚡ **Quick answer**
JFR is a low-overhead event recorder built into the JVM (free since Java 11). It captures CPU samples, allocations, GC, locks, I/O, exceptions and custom events with roughly 1% overhead, so it can run continuously in production and be analysed in JDK Mission Control.

What it answers in one recording: where CPU time goes (method profiling), what allocates the most, which locks threads wait on, how long GC pauses are, which exceptions are thrown, and slow socket/file I/O.

Terminal

```
# Start with the app (continuous, ring buffer on disk)
java -XX:StartFlightRecording=disk=true,maxage=6h,maxsize=500m,settings=profile -jar app.jar

# Or attach to a running JVM, record 2 minutes, dump to a file
jcmd 4321 JFR.start name=incident duration=120s filename=/tmp/incident.jfr settings=profile

# Quick CLI summary without a GUI
jfr summary /tmp/incident.jfr
jfr print --events jdk.GarbageCollection,jdk.JavaMonitorEnter /tmp/incident.jfr | head
```

💡 **Pro tip**
JFR event streaming (Java 14+) lets you consume events live from inside the app or a sidecar, so you can alert on long GC pauses or lock contention without external agents.

🔥 **Q337** JV-033 • Advanced

A Java service is at 100% CPU. How do you find the cause?

⚡ **Quick answer**
First decide whether it's the **application** or the **GC**: check GC logs or `jstat -gcutil`. If it's application code, find the hot threads (`top -H -p <pid>`), match their IDs to a thread dump, and confirm with a profiler (async-profiler or JFR).

Usual suspects once you find the hot frames:

- Infinite or very long loops (often from bad input or a race on a non-thread-safe `HashMap`).

© TechTribers. For personal use only. Do not redistribute.

- Catastrophic regex backtracking.
- Excessive logging or JSON serialisation in hot paths.
- GC thrashing because the live set is close to `-Xmx` (looks like CPU, is actually memory).

Terminal

```
# 1. Is it GC? Columns: S0 S1 E O M CCS YGC YGCT FGC FGCT ... GCT
jstat -gcutil 4321 1000 5            # FGC climbing + O near 100% = GC thrash, not app code

# 2. Which threads burn CPU?
top -H -p 4321                       # note thread id, e.g. 11343
printf '%x\n' 11343                  # -> 2c4f (hex)
jcmd 4321 Thread.print | grep -A 15 'nid=0x2c4f'

# 3. Profile properly (flame graph)
./asprof -d 30 -e cpu -f /tmp/cpu.html 4321
```

⚠️ **Interview trap**

Restarting the pod destroys the evidence. Capture `jstat`, two or three thread dumps and a short profile first. It takes about a minute.

📦 **Q338** JV-034 • Advanced

How does the JVM behave in containers? What is OOMKilled vs OutOfMemoryError?

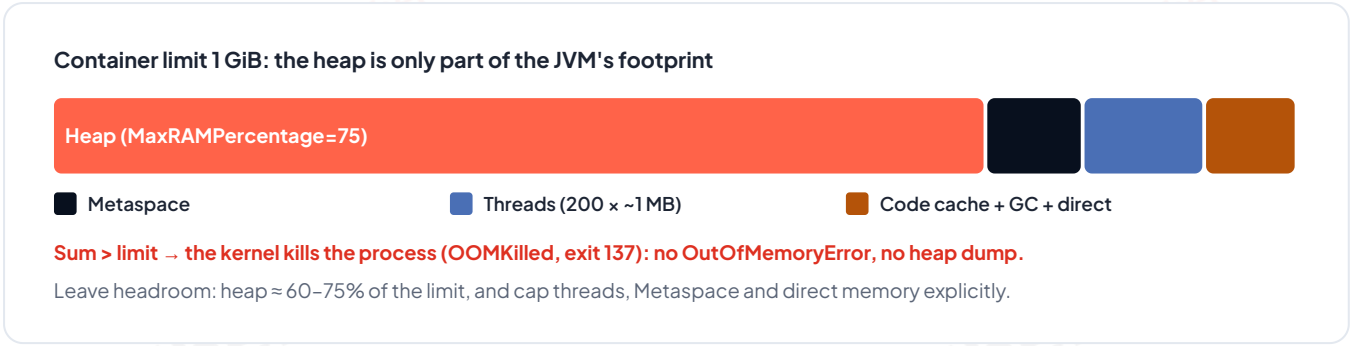
⚡ **Quick answer**

Since Java 10 (and 8u191) the JVM reads cgroup CPU and memory limits. By default the max heap is only 25% of the container limit; set `-XX:MaxRAMPercentage` (often 60–75%). If total process memory exceeds the limit, the kernel kills it (**OOMKilled**, exit 137) with no Java error at all.

	OutOfMemoryError	OOMKilled
Who decides	The JVM	The Linux kernel (cgroup limit)
Cause	A JVM memory area is full (heap, Metaspace…)	Total process RSS exceeds the container limit
Evidence	Stack trace, optional heap dump	Exit code 137, `OOMKilled` in `kubectl describe pod`
Usual fix	Fix leak or grow heap	Lower heap %, bound threads/Metaspace/direct memory, or raise the limit

© TechTribers. For personal use only. Do not redistribute.

```
Terminal
java -XX:+PrintFlagsFinal -version | grep -E "MaxHeapSize|ActiveProcessorCount"
java -XshowSettings:system -version      # shows the container limits the JVM detected
# Native memory breakdown (start with -XX:NativeMemoryTracking=summary)
jcmd 4321 VM.native_memory summary
```



⚠️ **Interview trap**

Setting `-Xmx` equal to the container limit guarantees OOMKills under load, because thread stacks, Metaspace, code cache, GC structures and direct buffers all need memory outside the heap.

Q339 JV-035 • Intermediate

What is the difference between a heap dump, a thread dump and a core dump?

⚡ **Quick answer**

A **thread dump** is a text snapshot of every thread's stack and lock state (cheap, seconds). A **heap dump** is a binary snapshot of all objects on the heap (expensive, pauses the JVM). A **core dump** is the OS-level image of the whole process memory, used for JVM crashes and native issues.

	Thread dump	Heap dump	Core dump
Answers	What is every thread doing?	What is using memory?	Why did the JVM process crash?
Capture	`jcmd PID Thread.print`, `kill -3`	`jcmd PID GC.heap_dump`, on OOM	OS on crash (`ulimit -c`), `gcore`
Cost	Milliseconds	Pause + file ≈ used heap	File ≈ whole process memory
Analyse with	Text editor, fastThread	Eclipse MAT, VisualVM	`jhsdb`, gdb; plus `hs_err_pid.log`

💡 **Pro tip**

When the JVM itself crashes (SIGSEGV), it writes `hs_err_pid<pid>.log` in the working directory. Read the "Problematic frame" line first: it usually points at a native library, JNI code or a JIT compiler bug.

© TechTribers. For personal use only. Do not redistribute.

Series progress

339 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 8 starts at Q340.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

TechTribers techtriber.com

© TechTribers. For personal use only. Do not redistribute.